

Multi-objective crowd-aware robot navigation system using deep reinforcement learning

Chien-Lun Cheng^a, Chen-Chien Hsu^{a,*}, Saeed Saeedvand^a, Jun-Hyung Jo^b

^a Department of Electrical Engineering, National Taiwan Normal University, Taipei, Taiwan

^b Department of Information and Communication Technology, Griffith University, Gold Coast, Australia

HIGHLIGHTS

- Deep reinforcement learning helps achieving robot navigation in a crowd environment.
- Crowd-aware robots can learn to reach the goal while balancing multiple objectives.
- Dual-selection attention module allows robots to reduce hesitation during navigation.
- Multi-reward function enables robots to learn to efficiently and effectively reach the goal.

ARTICLE INFO

Keywords:

Deep Reinforcement Learning
Human Aware Motion Planning
Human-Robot Interaction
Obstacle Avoidance

ABSTRACT

Navigating efficiently and safely through human crowds is essential for mobile robots in diverse applications such as delivery services, home assistance, healthcare, and manufacturing. However, traditional navigation methods are adversely affected by the high randomness of human movements, seriously hindering robot navigation in crowd environments. To tackle these problems, this paper proposes a deep reinforcement learning-based multi-objective crowd-aware robot navigation system called Multi-Objective Dual-Selection Reinforcement Learning (MODSRL). To deal with multiple objectives, including safety, time efficiency, collision avoidance, and path smoothness during navigation, a set of reward functions is used in MODSRL. To address the challenge of hesitation at the beginning when navigating in a crowd environment, a Dual-Selection Attention Module is developed, which enables the robot to make efficient decisions while reducing hesitation. Experimental results demonstrate that the proposed MODSRL outperforms existing approaches in terms of five different metrics. In particular, the average success rate of the proposed MODSRL outperforms ERVO, CADRL, LSTM-RL, and OM-SARL algorithms by 17.1%, 26.4%, 16.6%, and 9.2%, respectively, sufficing to show its robustness in complex crowd environments.

1. Introduction

With the advancement in robotics and artificial intelligence technologies, the rapid development of autonomous mobile robots (AMR) has garnered significant attention. These robots possess high elasticity and multi-purpose autonomous capabilities that align well with market needs, resulting in numerous applications in automated factories [1]. In addition to industrial settings, service robots [2] equipped with autonomous navigation capabilities have also become increasingly prevalent in human-robot coexistence environments such as airports and shopping malls. It has become evident that, whether for industrial purposes or home services, mobile robots need to possess autonomous navigation

capabilities to meet the requirements to accomplish complex tasks.

Over the past decades, autonomous navigation has been extensively researched with the aim of enabling robots to operate independently in unknown or partially unknown environments. To meet the navigation requirements, robot systems have to possess capabilities such as environment map building, localization, path planning, motion control, and obstacle avoidance. Fig. 1 illustrates a traditional robot navigation framework where Simultaneous Localization and Mapping (SLAM) [3], through sensors like LiDAR or cameras, is used to create a map of the surrounding environment. Localization algorithms like Adaptive Monte Carlo Localization (AMCL) [4] are employed to determine the robot's position within the map, enabling the planning of an obstacle-free path

* Corresponding author.

E-mail address: jhsu@ntnu.edu.tw (C.-C. Hsu).

<https://doi.org/10.1016/j.asoc.2023.111154>

Received 3 June 2023; Received in revised form 16 October 2023; Accepted 11 December 2023

Available online 13 December 2023

1568-4946/© 2023 Elsevier B.V. All rights reserved.

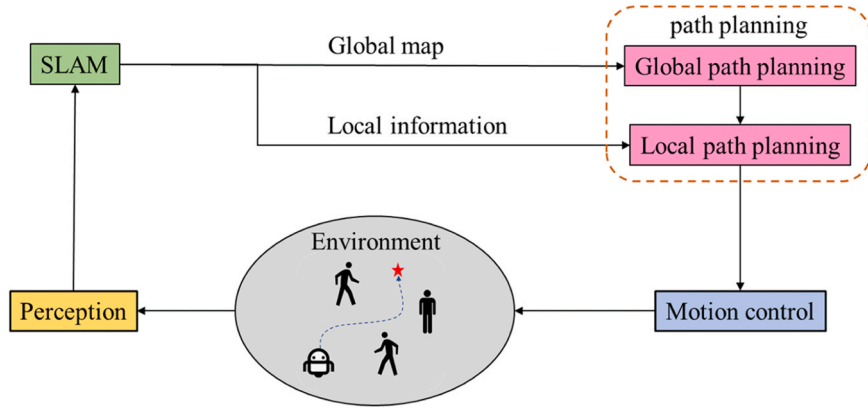


Fig. 1. Traditional robot navigation architecture.

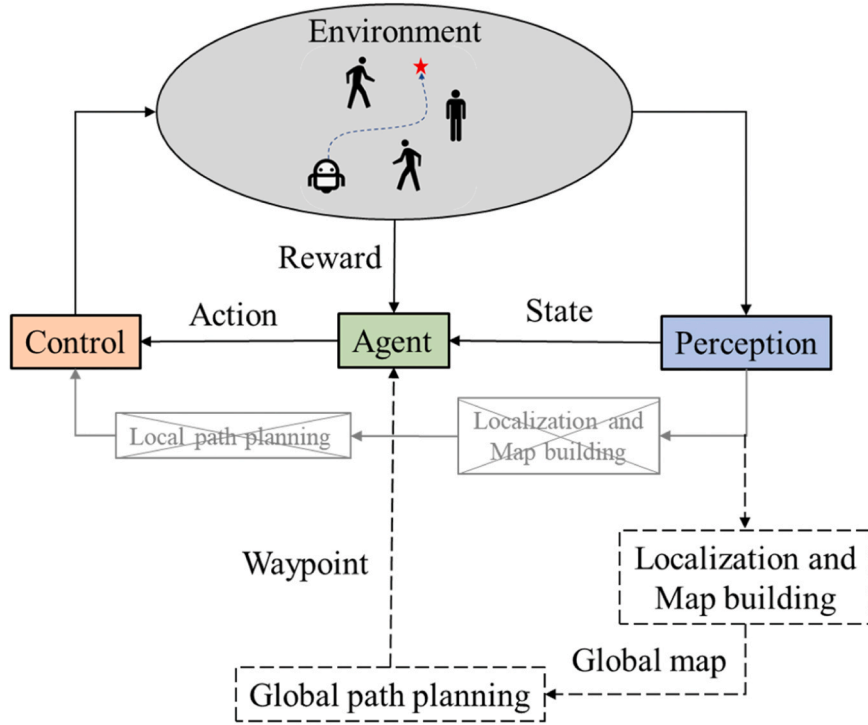


Fig. 2. Deep reinforcement learning-based navigation system [18].

to navigate to the target point.

Deep reinforcement learning represents a highly promising approach in the realm of artificial intelligence by leveraging the power of deep neural networks to approximate nonlinear functions so as to effectively tackle high-dimensional problems and handle large amounts of input information. This has led to significant improvements over traditional reinforcement learning, particularly in the field of navigation. As a result, deep reinforcement learning-based navigation has been used to replace or integrate with traditional navigation frameworks. Fig. 2 illustrates the interaction processes between an agent and the environment in a typical deep reinforcement learning-based navigation framework, where the agent takes on the roles of robot localization, map building, and local path planning typically found in traditional navigation architectures. Based on deep reinforcement learning, navigation systems have been designed to deal with various scenarios, such as local obstacle avoidance, indoor navigation, multi-robot navigation, and social navigation [5–7]. While these scenarios all involve navigation tasks, they each have their own unique aspects and details that need to be

considered. One method that has shown promising results in improving navigation performance in crowd environments is Socially Attentive Reinforcement Learning (SARL) [8]. SARL considered both human-robot and human-human interactions, which sets it apart from previous methods like CADRL [9], SA-CADRL [10], and LSTM-RL [11], which only considered one-way interaction between humans and robots. While SARL has been successful in improving navigation performance in crowd environments, it still has several shortcomings. First, it did not consider the reciprocal influence of the robot on humans because the impact of the robot on the humans will also affect the robot's further state. Second, the self-attention mechanism used in SARL may not be able to handle many agents in the environment, leading to a decrease in the navigation success rate. Third, using Optimal Reciprocal Collision Avoidance (ORCA) [12] as the crowd environment simulation algorithm ignored the emotional reactions of humans, which can lead to safety issues. Last, SARL did not consider the trade-off between safety, time efficiency, and path smoothness during navigation.

As an attempt to address the above-mentioned issues, this paper

proposes a new approach called Multi-Objective Dual-Selection reinforcement learning (MODSRL), developed on the basis of [13], [14], and [15], aiming to overcome the shortcomings of the previous works and achieve high performance in navigation tasks. First, we expand the single-layer attention module of SARL to two layers and embed a selection mechanism [13,14] in each layer to achieve a high attention score. As a result, it will improve the efficiency of robot navigation and reduce delays while considering the interaction of the robot with humans. Second, to consider the emotional characteristics of human actions, the concept of ORCA is changed to Emotional Reciprocal Velocity Obstacles (ERVO) [15] to make the simulated environment closer to the real scene, increasing the degree of safety. Third, we design a multi-reward function to create a multi-objective agent capable of learning to reach the goal while maintaining safety, time efficiency, collision avoidance, and path smoothness during navigation. In summary, this paper has the following contributions:

- I. To the knowledge of the authors, this is the first work where several reward functions are designed to create a multi-objective agent capable of learning to reach the goal while considering safety, time efficiency, collision avoidance, and path smoothness during navigation.
- II. A dual-selection-based social attention reinforcement learning algorithm is developed to reduce the hesitation at the beginning during navigation in crowd environments and enable robots to make better decisions efficiently.
- III. Experimental results show that the proposed MODSRL achieves higher performance and robustness than the previous studies in terms of success rate, collision rate, and timeout rate with the increasing number of humans in the environment.

2. Literature review

2.1. Traditional methods

As illustrated in Fig. 1, various traditional methods were revealed for mobile robot navigation. Among them, [16] used a large dataset of obstacles, open areas, and pedestrian images, applying a progressive Bayesian learning technique to classify corridor features and construct an environmental map using these features. [17] used laser and camera sensors to accomplish 3D mapping, where the camera image information is then employed for training a decision classifier through the Random Forests Method to achieve localization. Subsequently, navigation functions such as path planning were developed using the Rapidly Exploring Random Tree (RRT). While the aforementioned methods enable robot navigation in various environments, traditional navigation methods often come with significant computational errors. These errors gradually accumulate across mapping, localization, path planning, and obstacle avoidance processes, resulting in suboptimal performance in practical applications. Moreover, due to high reliance on sensor accuracy, the ability to handle unknown or dynamic environments is also limited [18]. For crowd environments, traditional navigation algorithms struggle to cope with the constantly changing movements and interactions of humans, which can make the situation worse and even lead to potential dangers.

Basically, traditional crowd navigation methods can be divided into two categories: reaction methods and trajectory methods. Reaction methods generally use one-step interaction rules to obtain the robot's optimal action. For example, Reciprocal Velocity Obstacle (RVO) [19] and ORCA [12] utilized linear programming to find collision-free velocities by intersecting semi-planes. Social Force Models (SFM) [20] utilized attractive and repulsive forces to model interactions between the robot and other agents in crowd environments. These algorithms considered local interactions and neighboring information to generate collision-free trajectories. However, reaction methods generally do not consider the future states of the neighboring agent and use only

hand-crafted functions to safeguard safety, which may lead to immature actions and short-sighted perception, resulting in safety problems and hindering robot navigation in crowd environments. On the other hand, trajectory methods primarily use a data-driven approach, such as Social LSTM [21] and Social GAN [22], to predict future trajectories of humans to plan safe paths. However, the highly random actions of humans in crowd environments increase calculation costs and complexity, which might cause the freezing robot problem (FRP) [23] and perform unnecessary actions to avoid collisions. These issues have made the trajectory methods unsuitable for robot navigation in crowd environments.

2.2. Deep reinforcement learning methods

In reinforcement learning, any decision-maker is represented as an agent, while the environment encompasses everything external to this agent. The agent is then interacting with its environment to maximize cumulative rewards. However, reinforcement learning faces limitations, particularly in high-dimensional state scenarios. While offering a promising approach to train robots for navigation tasks, reinforcement learning encounters challenges when confronted with intricate environments, especially those involving crowded spaces. As a result, deep reinforcement learning emerged to address these challenges using deep neural networks. This evolution reflects the desire to overcome reinforcement learning limitations, enabling effective robot navigation in complex, crowded environments, as shown in Fig. 2. Thus, various learning methods, such as [9], [10], and [11], have been utilized to achieve crowd-aware navigation in social model navigation research. These methods can cooperate with humans but fail to deduce complex human-robot interactive behaviors. To address this issue, SARL was proposed, using imitation learning for training and the ORCA strategy for model initialization. Additionally, SARL featured a Social-Attentive pooling model that extracts the cooperative behavior of crowds in a deep reinforcement learning framework. Although SARL enables robots to navigate in crowd environments while being socially aware, it still has some limitations. To address these limitations, DSARL [13] expanded the attention model in SARL into two layers, effectively improving the efficiency of robot navigation and reducing delay. [14] used mathematical methods to reduce the influence of attention scores on the robot, which allows the robot to make better decisions during navigation. However, these methods are not able to fully solve all the problems associated with SARL. To address the safety issue in crowd-aware robot navigation with learning methods, [24] proposed safety rules by considering high-level obstacle information to make robots navigate safely. This approach, however, did not consider the time efficiency and path smoothness. [25] considered the trade-off between safety and time efficiency based on [8]. Unfortunately, the attention model is not complex enough to deal with a large number of humans.

2.3. Multi-objective reinforcement learning

Robots in crowd environments need to navigate safely and efficiently while achieving multiple objectives, which may conflict with each other. To address this issue, designing multiple reward functions can help the robot achieve a good balance between different objectives, such as reaching the destination in time, following a smooth path, and avoiding collisions. Thus, multi-objective approaches aim to make trade-offs and improve the robot's overall performance in the hope of finding more effective and efficient solutions in complex scenarios. To attain solutions that cater to multiple objectives, previous studies revealed that multiple sets of reward functions could be designed and combined into a single objective if these objectives are independent or directly related [26,27]. Among them, [28] designed reward functions to simultaneously achieve the objectives of reaching goals, crossing areas, and avoiding collisions. Moreover, KAI-DDPG [29] also incorporated a multi-objective reward function and introduced the concept of information entropy to dynamically adjust the coefficients of the reward function, aiming to enhance

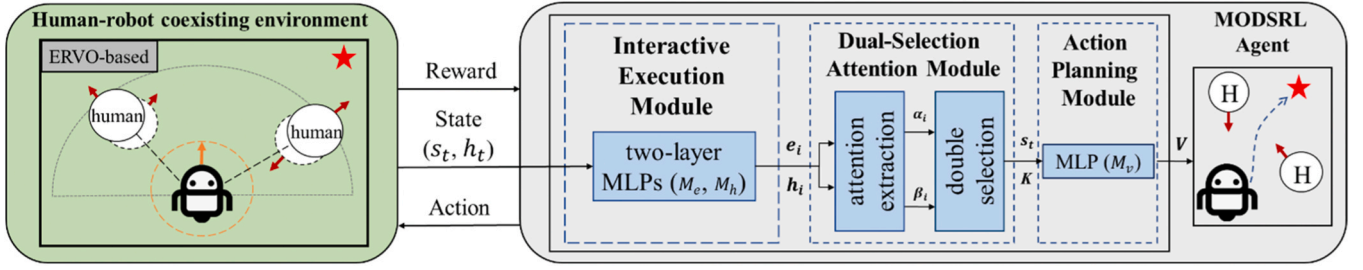


Fig. 3. Overall framework of the proposed MODSRL.

navigation efficiency. In [30], the authors proposed an interactive model predictive control framework to predict human intentions and their interactions so that a multi-objective function is designed to optimize the robot trajectory. In this paper, we will design multiple reward functions, each of which can be treated as an objective. According to the above methods, we design several reward functions to enable the robot to navigate safely and efficiently in crowd environments while considering multiple objectives such as timeliness, path smoothness, and collision avoidance.

3. Problem formulation

In the framework of deep reinforcement learning, the robot navigation task is regarded as a sequential decision-making problem. The interaction process between the agent and environment can be modeled as a Markov Decision Process (MDP), which includes four main components: state of the environment S , action A taken by the agent, reward R that the agent receives for its actions, and state transition probability P .

For robots to navigate in a crowd environment, we assume that the robot can perceive the state information of humans and the robot itself from sensors such as lidar or depth cameras. The state vector h_t of each human and the state vector s_t of the robot itself at time t are defined as follows:

$$s_t = [d_g, v_{pref}, v_{linear}, v_{angle}, r]$$

$$h_t = [p_x, p_y, v_x, v_y, r_i, d_i, r_i + r]. \quad (1)$$

Each agent (robot or human) can observe each other's position $P = [p_x, p_y]$, velocity $V = [v_x, v_y]$, and radius r , while the unobservable states are the preferred velocity v_{pref} and the distance between the robot and the target point d_g . The state vector h_t of each human includes its position P , velocity V , radius r , and the distance between the robot and the current human d_i . Finally, the state vector s_t of the robot and the state vector h_t of each human are combined into $s_t^{jn} = [s_t, h_t]$. The velocity of the robot v_{linear} was discretized into five speeds in $(0, v_{pref}]$ and the angular velocity $v_{angular}$ was discretized into eight orientations evenly from within $[0, 2\pi)$. In total, there are 41 dimensional action spaces (40 discrete actions and stationary). Reinforcement learning is then used to find an optimal policy $\pi^*(s_t^{jn})$, by maximizing the cumulation of the discounted total rewards, as shown in (2).

$$\pi^*(s_t^{jn}) = \underset{a_t}{\operatorname{argmax}} R(s_t^{jn}, a_t) + \gamma^{\Delta \bullet v_{pref}} \int P(s_t^{jn}, a_t, s_{t+\Delta t}^{jn}) V^*(s_{t+\Delta t}^{jn}) ds_{t+\Delta t}^{jn} \quad (2)$$

$$V^*(s_t^{jn}) = \sum_{i=1}^T \gamma^{i \bullet v_{pref}} R_i(s_t^{jn}, \pi^*(s_t^{jn})), \quad (3)$$

where $R(s_t^{jn}, a_t)$ is the reward received at each time, γ is the discount factor between zero and one, and $P(s_t^{jn}, a_t, s_{t+\Delta t}^{jn})$ is the state transition probability from time t to time $t + \Delta t$. v_{pref} is employed to standardize the discount factor, with the goal of achieving numerical stability [7].

$V^*(s_t^{jn})$ is the optimal value function that estimates the expected return as shown in (3), where T is the total number of decision steps from the state at the time t to the final state. We use a deep neural network, which we refer to as the value network, to estimate the value function V . To train the value network, we use the temporal-difference (TD) method and experience replay mechanism, so that the value function is corrected in the direction of TD error.

To make the robot achieve multi-objective effects, we design a multi-reward function (R_{mo}) related to safety, time-efficiency, collision avoidance, and path smoothness based on the reward functions of [8, 15], as shown below:

$$R_{mo} = [r_{safety}, r_{time}, r_{collision}, r_{smooth}] \quad (4)$$

$$r_{safety} = \gamma(d_t - r^b), \text{ if } d_t < r^b \quad (5)$$

$$r_{time} = 1.0 - \alpha\left(\frac{t}{t_{limit}}\right), \text{ if } p_t = p_{goal} \quad (6)$$

$$r_{collision} = \begin{cases} r_c, & \text{if } d_t < 0 \\ \beta(d_t - d_{discomfort}), & \text{if } d_t < d_{discomfort} \end{cases} \quad (7)$$

$$r_{smooth} = \varepsilon(\omega_{th} - \omega_t), \text{ if } \omega_t > \omega_{th} \quad (8)$$

In particular, r_{safety} uses the range of active path-clearing actions mentioned in [15], where r^b is the effective range of active path-clearing actions. We set r^b to be 1.5 m. If a human enters this range, the human will be affected by the robot, causing them to dodge the robot, making the scenario more realistic. d_t is distance between a robot and a human. r_{time} can make the robot reach the goal as quickly as possible, where t_{limit} is a fixed constant. $r_{collision}$ can be separated into two terms depending on d_t , r_c is a negative value indicating the penalty for collision, and $d_{discomfort}$ is the minimum distance between the human and the robot, which is set as 0.2 m. This setting can be used to avoid the minimum collision. r_{smooth} is designed to avoid the excessive rotation angle of the robot and shorten the overall path. Because the overall path is reduced, the overall navigation time will also be shortened, which further strengthens the time efficiency of the robot. We set ω_{th} to be 50 degrees. γ , α , β , and ε are parameters indicating the degree of influence of the respective reward functions, which are set to 0.2, 0.1, 0.5, 0.01, respectively.

To achieve the balance between safety, time efficiency, collision avoidance, and path smoothness objectives, a comprehensive evaluation method is necessary. In [25], the authors defined a danger frequency η , which is the ratio of the total number of times the robot enters a Danger Zone over the total navigation time. In this paper, we instead consider the danger frequency η as the frequency of humans entering the active path-clearing range as follows:

$$SE = 1 - \eta \quad (9)$$

$$TE = \frac{t_{avg} - t_{max}}{t_{min} - t_{max}} \quad (10)$$

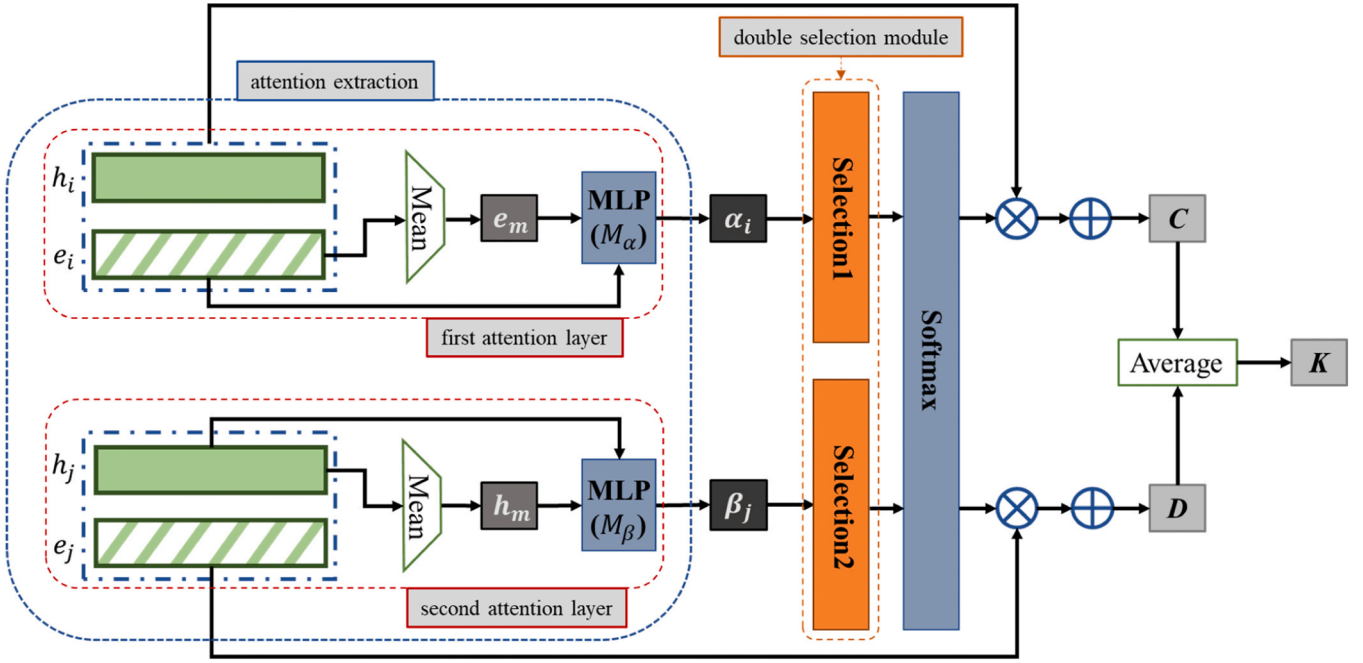


Fig. 4. Architecture of Dual-Selection Attention Module.

A higher value of the danger frequency η results in a lower safety-efficiency (SE), and vice versa. The time efficiency (TE) is defined as the ratio of the actual time taken to reach the goal by the robot over the shortest possible time. Here, $t_{\min}$ is defined as the travel distance divided by the maximum robot speed, and $t_{\max}$ is defined as the longest navigation simulation time. A higher value of TE indicates a greater efficiency.

4. Proposed approach

This paper proposes a multi-objective crowd-aware robot navigation system, MODSRL, based on deep reinforcement learning for robots navigating in a crowd environment. The system is designed to achieve multi-objective navigation solutions, including safety, time efficiency, collision avoidance, and path smoothness. With reference to Fig. 3, the overall framework of the proposed system as a value network consists of three main modules as follows.

- I. Interactive Execution Module: Receive joint state s_t^n containing the state of each human and the state of the robot itself during robot navigation, which serves as the input to two-layer multi-layer perceptions (MLPs) to obtain a fixed-length vector e_i and interaction features h_i .
- II. Dual-Selection Attention Module: Use the obtained fixed-length vector e_i and interaction features h_i from the Interactive Execution Module as the input to evaluate interactions between humans and robot. Through a double selection module, the more important attention scores α_i , β_i and average crowd vector K can be obtained.
- III. Action Planning Module: An estimated value V can be obtained through an MLP after obtaining the average crowd vector K according to the Dual-Selection Attention Module.

4.1. Interactive execution module

To evaluate the interaction between humans and robot during crowd navigation, it is necessary to consider the impact of each human on the robot, as well as the impact of humans on each other. To address this

requirement, we utilize two-layer MLPs in the Interactive Execution Module in Fig. 3 to process the complex human state information obtained from sensors. When humans pass by the robot, the robot receives the state $s_t^n = [s_t, h_t]$. Through the first MLP M_e with ReLU as the activation function, a fixed-length vector e_i can be obtained according to (11), where W_e is the weight of the MLP.

$$e_i = M_e(s_t, h_t; W_e) \quad (11)$$

This output e_i is then taken as the input to the second MLP M_h which also uses ReLU as the activation function, where W_h is the network weights. As a result, the interaction features h_i between the robot and each human can be obtained from (12).

$$h_i = M_h(e_i; W_h). \quad (12)$$

4.2. Dual-Selection attention module

As described in Section 3, the state space and action space have been explicitly defined. When the human number increases, the state h_t will become more complex. In order to process the information effectively, regardless of whether the number of humans changes, it is necessary to construct a module that can handle any number of humans in order to correctly estimate the interaction of each human's action on the robot. Fig. 4 illustrates the architecture of the dual-selection attention module, in which an attention extraction module containing two attention layers is used to obtain an attention score of each human so that the robot can know the humans who have a greater influence on it through the attention score. Second, we use a double selection module to filter the attention scores that have less impact on the robot.

In the first attention layer of the attention extraction module, e_i obtained from the interactive execution module is used as the input. As shown in (13), we take the arithmetic average to get e_m , where n is the number of e_i to e_j .

$$e_m = \frac{1}{n} \sum_{k=1}^j e_k \quad (13)$$

Then the fixed-length vectors e_i to e_j and e_m are used as the input to an MLP M_α which uses a LeakyReLU activation function, where W_α is its weight. Finally, the attention score α_i of each human can be obtained

from (14).

$$\alpha_i = M_\alpha(e_i, e_m; W_\alpha) \quad (14)$$

To improve the robot's decision-making process, a selection mechanism (Selection1) is developed, in which we calculate the mean μ_α and standard deviation σ_α for attention scores α_i according to (15) and (16), respectively.

$$\mu_\alpha = \frac{1}{n} \sum_{k=1}^n \alpha_k \quad (15)$$

$$\sigma_\alpha = \sqrt{\frac{1}{n} \sum_{i=1}^n (\alpha_i - \mu_\alpha)^2} \quad (16)$$

If the difference between μ_α and σ_α is greater than α_i , we set α_i to a negative value nv ; otherwise, we leave it unchanged according to (17). Next, we use the SoftMax function to normalize attention scores α_i for each human, and multiply the output by interaction features h_i . Finally, we obtain the weighted linear combination vector C by summing the multiplied results of each pair according to (18).

$$\alpha_i = \begin{cases} \alpha_i, & \text{if } \alpha_i \geq \mu_\alpha - \sigma_\alpha \\ nv, & \text{if } \alpha_i < \mu_\alpha - \sigma_\alpha \end{cases} \quad (17)$$

$$C = \sum_{i=1}^n \text{softmax}(\alpha_i) h_i \quad (18)$$

In the second attention layer, we first calculate the arithmetic mean of the interaction features h_i between the robot and each human to obtain the output h_m according to (19).

$$h_m = \frac{1}{n} \sum_{k=1}^n h_k \quad (19)$$

Then the interaction features h_i and h_m are used as the input to an MLP M_β in (20), which uses a LeakyReLU activation function to obtain an attention score β_i , where W_β is its weights.

$$\beta_i = M_\beta(h_i, h_m; W_\beta) \quad (20)$$

Similar to the first attention layer, we utilize another selection mechanism (Selection2) to enable the robot to make more accurate decisions. Unlike the first selection, if the arithmetic mean μ_β from (21) minus three times the standard deviation σ_β from (22) is greater than β_i , then it is set to negative infinity ($-\text{inf}$) according to (23). This setting is intended to avoid filtering out too many important features from interaction features h_i . Finally, the attention score β_i passed through SoftMax and multiply by e_i is summed together to obtain a vector D , as shown in (24).

$$\mu_\beta = \frac{1}{n} \sum_{k=1}^n \beta_k, \quad (21)$$

$$\sigma_\beta = \sqrt{\frac{1}{n} \sum_{i=1}^n (\beta_i - \mu_\beta)^2} \quad (22)$$

$$\beta_i = \begin{cases} \beta_i, & \text{if } \beta_i \geq \mu_\beta - 3\sigma_\beta \\ -\text{inf}, & \text{if } \beta_i < \mu_\beta - 3\sigma_\beta \end{cases} \quad (23)$$

$$D = \sum_{i=1}^n \text{softmax}(\beta_i) e_i \quad (24)$$

Finally, the output vector C from the first attention module and the output vector D from the second attention module are added and averaged to obtain a crowd vector K , which serves as the input to the action planning module.

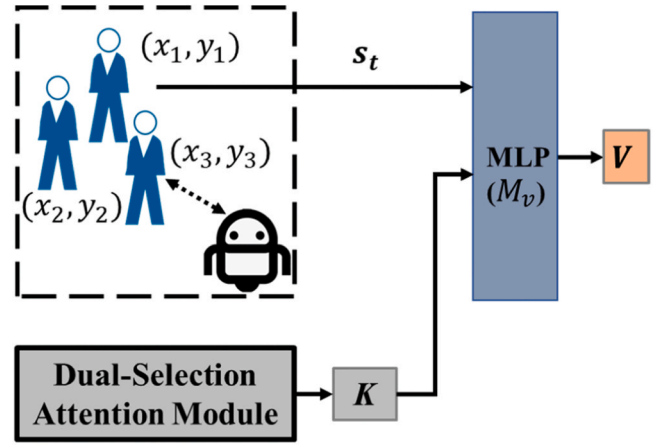


Fig. 5. Architecture of the action planning module.

4.3. Action planning module

After obtaining the vector K from the Dual-Selection Attention Module, it is expected to use the action planning module to obtain an estimated value V for evaluating the motion state. As shown in Fig. 5, an MLP M_v with ReLU as the activation function and W_v as its weight uses the state vector s_t and vector K as inputs to obtain the estimated value V as shown in (25). Through deep reinforcement learning, V is optimized to obtain an optimal value V^* , based on which we can obtain an optimal robot navigation strategy.

$$V = M_v(s_t, K; W_v). \quad (25)$$

5. Experimental results

In this section, we will elaborate on the simulation environment and provide a comprehensive presentation of our experimental results from the simulation.

5.1. Simulation Environment and Experiment Settings

The 2D simulation environments in this paper are adopted from [8] and [15] for evaluating the performance of various approaches. Fig. 6 shows a comparison of the training results based on the ORCA and ERVO methods where the agents can react when facing danger, in which Fig. 6. (a) shows that the overall collision rate based on the ERVO method is lower than ORCA before 4000 episodes, and Fig. 6. (b) shows that the average navigation time to reach the goal increases due to safety considerations. Thanks to the characteristics of ERVO, the collision rate between humans and robot is decreased, thereby enhancing safety. However, the improved safety performance by ERVO results in a slightly longer navigation time. To address this issue, we plan to use a time-efficiency reward function r_{time} to ensure that the robot maintains both safety and time-efficiency during crowd navigation.

Thus, in our simulation environment, the experimental setting is a circle with a radius of 4 m, where humans and the robot were positioned along its circumference. Two types of environments are considered in this paper, both of which involve simulated robots and humans with a radius set to 0.3 m. In the fixed environment, the preferred velocity of the robot and humans is set to 1 m/s. In the random environment, humans randomly select a desired speed within the range of 0.75 m/s to 1.25 m/s. After reaching their individual goals, humans in the random environment randomly choose another non-overlapping goal to navigate towards, avoiding any overlap with the goals of existing humans and robot. We use unicycle kinematics for the robot, which can be the reduced model of differential kinematics. All humans are controlled by ERVO, and the number of humans is varied to increase the complexity of

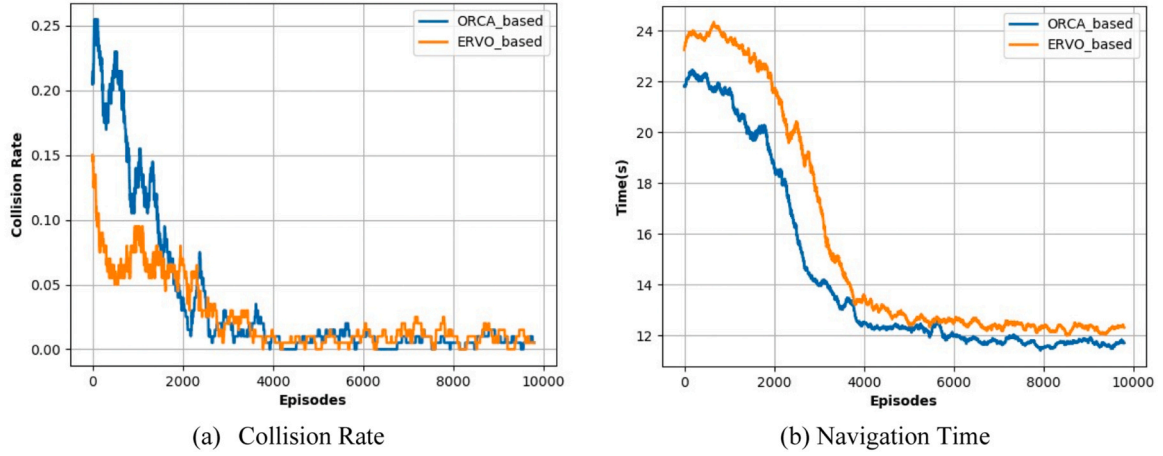


Fig. 6. Training results based on collision rate and navigation time of ORCA and ERVO.

the environment. Specifically, we set the number of humans to 5, 10, and 15, respectively. Also, we use the invisible setting [8] to prevent the system learning from aggressively achieving high rewards. By doing so, humans can only interact with other humans but not with the robot. Note that the invisible setting of the simulation environment is different from reality, where the motions of humans and those of the robot mutually affect each other. This limitation, however, can be solved to some extent by ERVO because it makes humans in the environment responsive to robots when facing danger.

We train the value network in Fig. 3 for 10 K episodes, where the first 3k for imitation learning from ERVO are used to train the policy 50 epochs with a learning rate of 0.01 using an Adam optimizer. For the RL part, we set the learning rate to 0.001 and the discount factor to 0.9. The epsilon-greedy policy's exploration rate starts at 0.5 and decreases linearly to 0.1 over the first 5000 episodes. It then remains at 0.1 for the remaining 5000 episodes. The models are tested using 500 randomly generated environments. The parameters used in the value network include $M_e(13, 150, 100, 100)$, $M_i(100, 100, 50, 50)$, $M_r(200, 100, 100, 1)$, $M_\beta(100, 100, 100, 1)$, $M_v(56, 150, 100, 100, 1)$, which respectively represent the input, hidden, and output layers with varying widths in each MLP. In particular, the model was trained on an NVIDIA RTX 2060 GPU. It took an average of 20 to 24 h for the MODSRL to converge, depending on the number of humans ranging from 5, 10, and 15.

5.2. Baselines and ablation models

We compare the performance of our proposed method against two different categories: traditional methods using ERVO, SFM [20], and deep reinforcement learning methods, including CADRL [9], LSTM-RL [11], and OM-SARL [8]. To evaluate whether the selection part has a time-reducing effect on navigation delay, we remove the selection module and call it MODRL as an ablation model. To arrive at a fair

comparison, we incorporate the proposed multi-reward function into CADRL, LSTM-RL, and OM-SARL and use ERVO for imitation learning while keeping the overall network architecture the same. Finally, we use five metrics to evaluate the performance of the above methods, namely success rate, collision rate, timeout rate, navigation time, and distance, which are defined as follows:

- I. Success Rate: The rate at which the robot safely reaches the goal during the navigation process.
- II. Collision Rate: The rate at which the robot collides with humans during the navigation process.
- III. Timeout Rate: The rate at which the navigation process exceeds the simulation time, which is set to 25 s. The combined value of the success rate, collision rate, and timeout rate is equal to one.
- IV. Navigation Time: The average time taken by the robot to successfully reach the goal during the navigation process.
- V. Navigation Distance: The average distance traveled by the robot to successfully reach the goal during the navigation process.

5.3. Quantitative evaluation

The results of robot navigation in crowd environments for various methods are shown in Table 1. Compared to the traditional method in environments with 5, 10, and 15 humans, the MODSRL outperforms the ERVO and SFM in all five metrics. Note that as the number of humans increases, the success rate of ERVO and SFM gradually decreases while the collision rate and timeout rate gradually increase. These findings suggest that the traditional method experiences a freezing robot problem in denser crowd environments, as shown in Fig. 7. (a). With MODSRL, the robot can reach the goal with efficiency and safety while keeping a straight trajectory, as shown in Fig. 7. (b).

Compared to the learning methods, CADRL* is better than OM-

Table 1

Navigation results for different numbers of humans in a fixed environment.

Method	Success Rate ↑			Collision Rate ↓			Timeout Rate ↓			Navigation Time ↓			Navigation Distance ↓		
	Human Numbers														
	5	10	15	5	10	15	5	10	15	5	10	15	5	10	15
SFM[20]	0.778	0.416	0.056	0.116	0.128	0.158	0.106	0.456	0.786	17.28	20.88	22.63	11.06	11.21	12.03
ERVO[15]	0.904	0.802	0.604	0.080	0.136	0.190	0.016	0.062	0.206	12.20	15.60	17.91	9.31	10.48	11.28
CADRL*[9]	0.860	0.734	0.438	0.120	0.242	0.552	0.020	0.024	0.010	11.27	13.80	15.55	8.44	8.66	8.56
LSTM-RL*[11]	0.968	0.832	0.524	0.006	0.060	0.108	0.026	0.108	0.368	12.68	16.01	18.03	8.80	10.92	13.78
OM-SARL*[8]	0.924	0.878	0.746	0.020	0.050	0.104	0.055	0.072	0.150	11.60	14.42	16.89	8.78	10.41	12.65
MODRL (Ours)	0.992	0.914	0.790	0.004	0.048	0.110	0.004	0.038	0.100	11.91	14.37	15.04	8.40	10.73	11.07
MODSRL (Ours)	0.998	0.950	0.874	0.002	0.034	0.078	0.000	0.016	0.048	11.14	14.13	15.15	8.78	10.19	10.77

*:Modified to incorporate the proposed multi-reward function

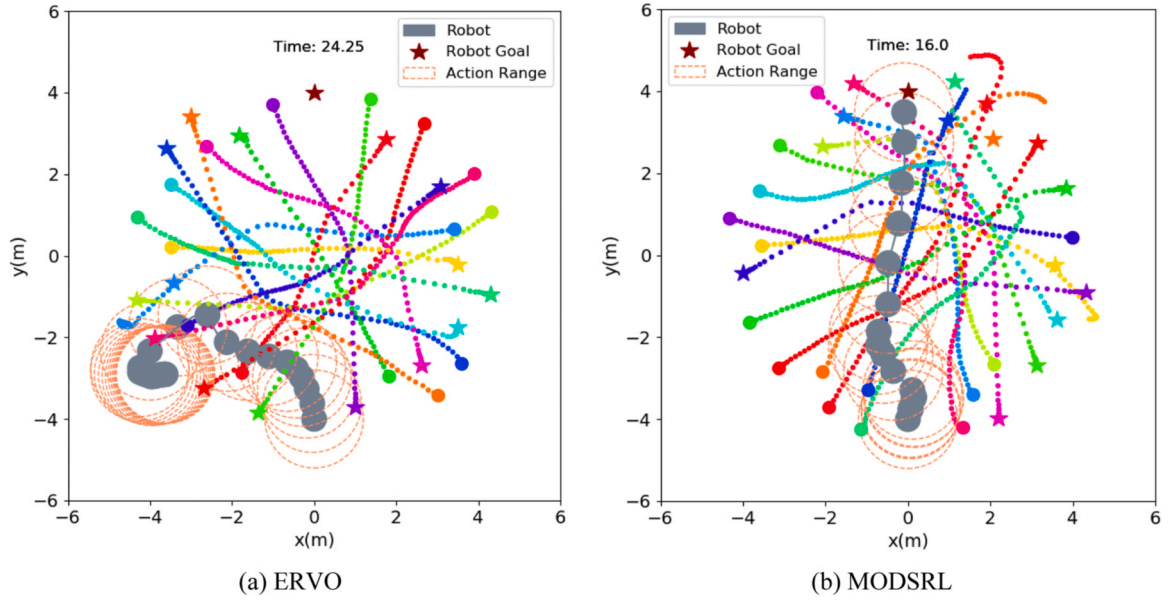


Fig. 7. Navigation trajectories of ERVO and MODSRL with 15 humans. The dark red star is the navigation goal, the other colored stars are human goals, and the orange circle represents the active path-clearing action range [15].

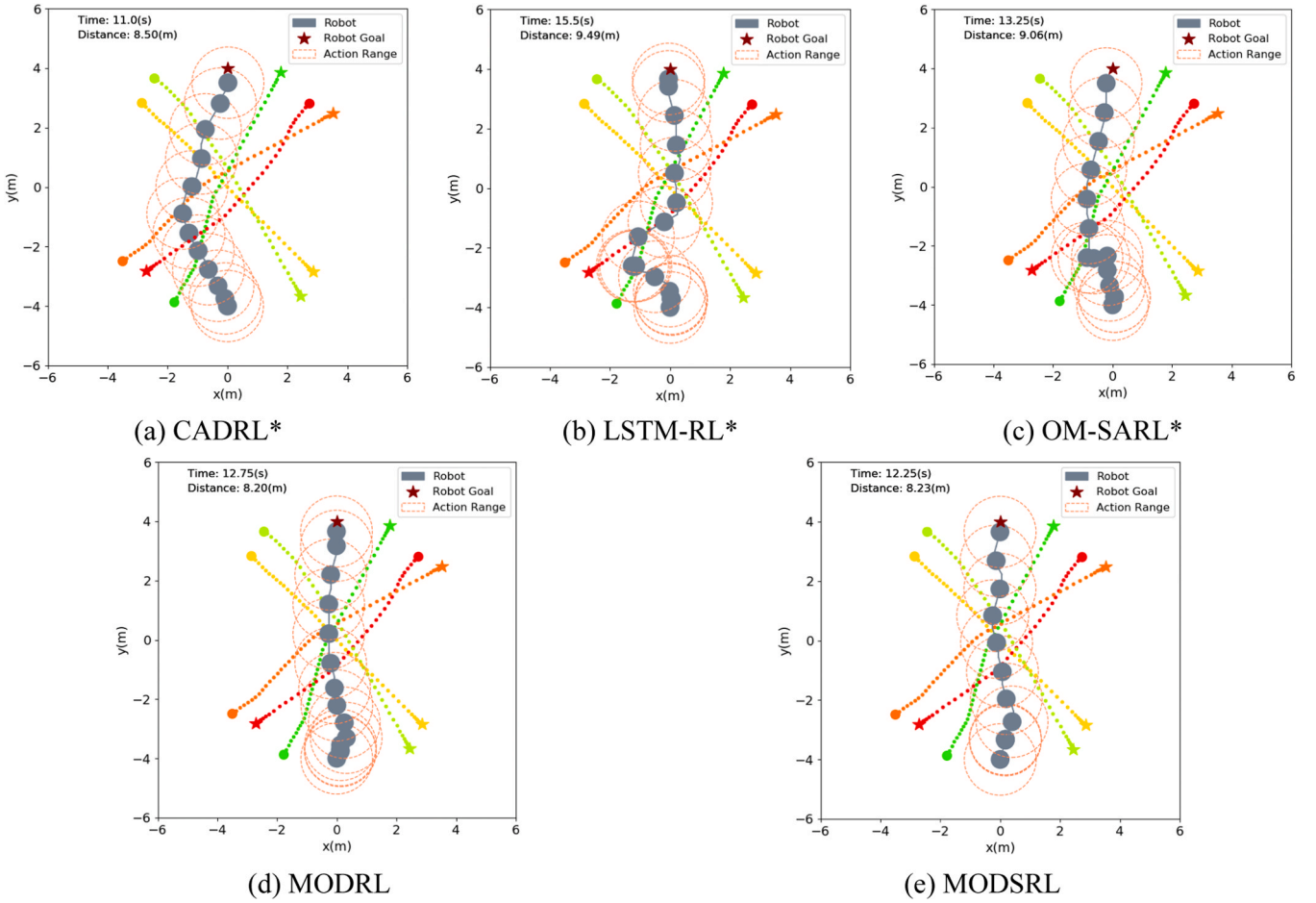


Fig. 8. Navigation trajectories of five learning methods with five humans in the same environment.

SARL* and LSTM-RL* in terms of overall navigation distance and time in environments with different numbers of humans, but the collision rate is the highest. MODSRL has the best success rate, collision rate, and

timeout rate compared to other baselines in environments with 5 and 10 humans. However, in an environment with 15 humans, CADRL* has the best timeout rate, and MODSRL ranks second. As a result, CADRL* has

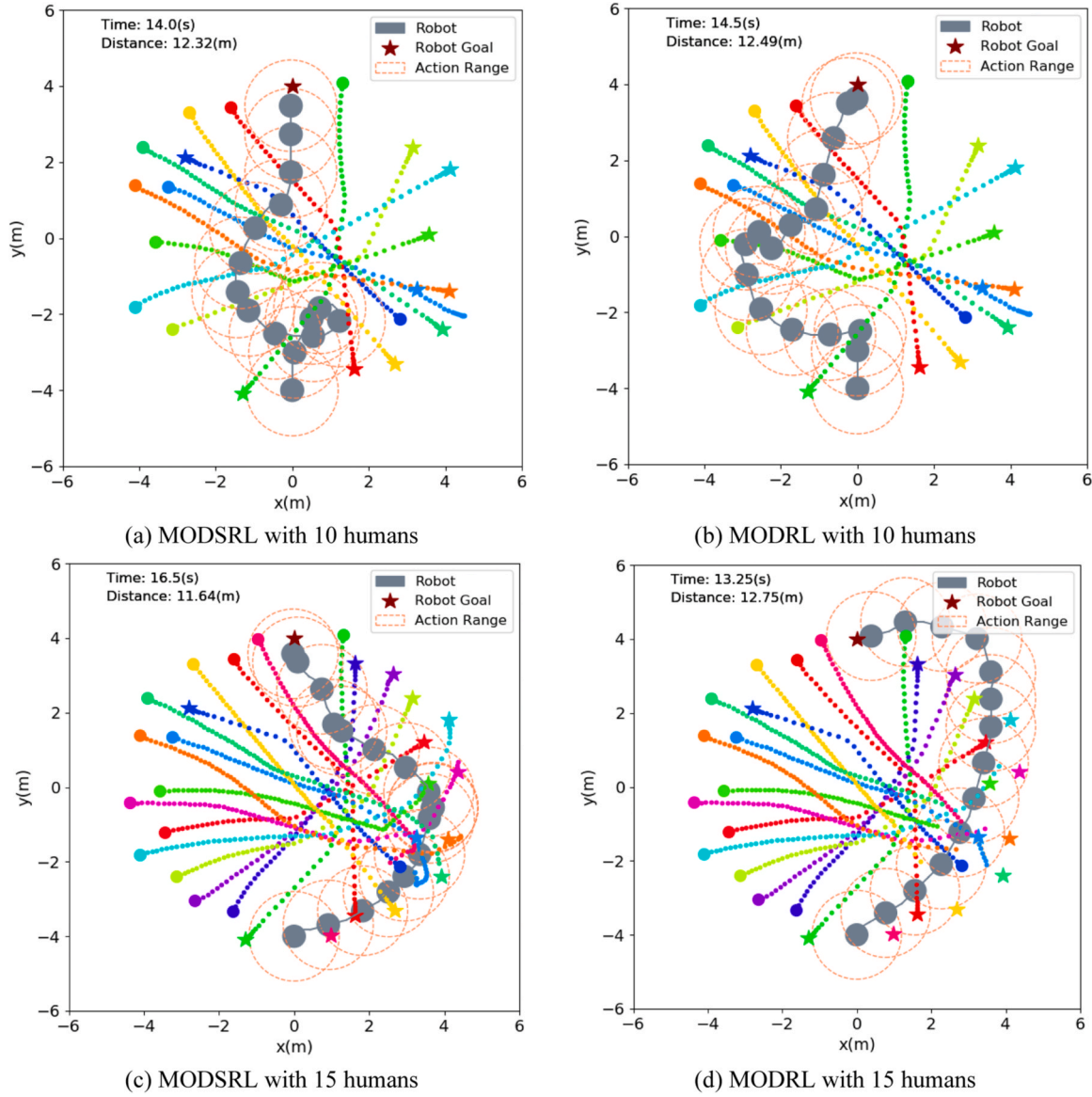


Fig. 9. Navigation trajectories of MODSRL and MODRL with 10 and 15 humans.

the shortest navigation distance and better navigation time but at the expense of safety. MODSRL outperforms CADRL* by 43.6% in success rate and has a 47.4% lower collision rate with 15 humans, as shown in Table 1, showing that the robot is better able to reach the goal while maintaining safety. Fig. 8 shows the navigation trajectories of the five methods, among which LSTM-RL*, OM-SARL*, and MODRL experience delays at the beginning of navigation, as shown in Fig. 8. (b) – (d). Although CADRL* quickly finds a path and has good performance in navigation time and distance, as shown in Fig. 8. (a), the collision rate shown in Table 1 is greater than the other methods, resulting in safety concerns. MODSRL, on the other hand, can quickly plan the path and avoid the robot wandering at the beginning while keeping a straight line until reaching the goal, as shown in Fig. 8. (e).

Compared to the ablation model MODRL, MODSRL shows superior performance in success rate, collision rate, and timeout rate. In the environment with five humans, MODSRL has a shorter navigation time of 0.77 s in comparison to MODRL but a longer navigation distance of 0.38 m, as indicated in Table 1. This is because the inclusion of the selection mechanism in MODSRL can help robots reduce less important interactions with humans, make better decisions, and reduce initial motion delay, but it may slightly increase the distance traveled, as

shown in Fig. 8. (d) and Fig. 8. (e). In the environment with ten humans, MODSRL outperforms MODRL in both navigation time and distance, as shown in Fig. 9. (a) and Fig. 9. (b). In the environment with 15 humans, however, the navigation time via MODSRL increases by 0.11 s compared to MODRL, but the navigation distance decreases by 0.30 m, as shown in Table 1. As shown in Fig. 9. (c) and Fig. 9. (d), the navigation time of MODSRL is higher than MODRL, but the navigation distance is shorter than MODRL. These results demonstrate that MODSRL achieves the best success and collision rates, regardless of the number of humans in the environment, making robot navigation more efficient. Furthermore, as shown in Fig. 10, the proposed MODRL and MODSRL can learn to maximize rewards, considering safety, time efficiency, collision avoidance, and path smoothness objectives for robot navigation. In the environment with five humans, due to the double selection module, MODSRL achieves a higher value of 0.0186 for the maximum total reward, as shown in Fig. 10. (a) and Fig. 10. (c). Furthermore, in the environment with ten humans, MODSRL not only obtains a higher value of 0.0135 for the maximum total reward but also exhibits improved training results, as shown in Fig. 10. (b) and Fig. 10. (d).

To evaluate the performance of MODSRL in terms of safety and time efficiency using the method [20] described in Section 3, we calculate the

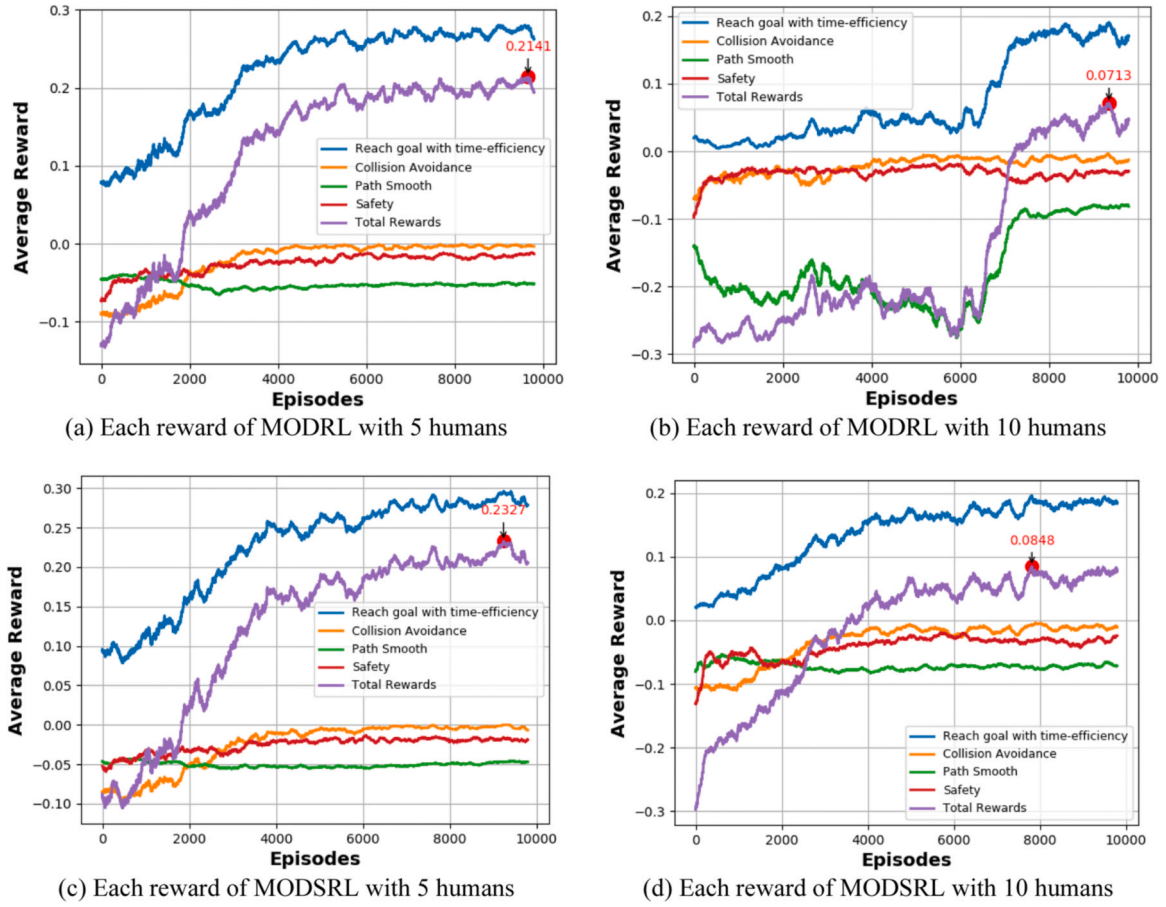


Fig. 10. Training results of each reward where both MODRL and MODSRL learn to maximize several rewards in 10 K episodes.

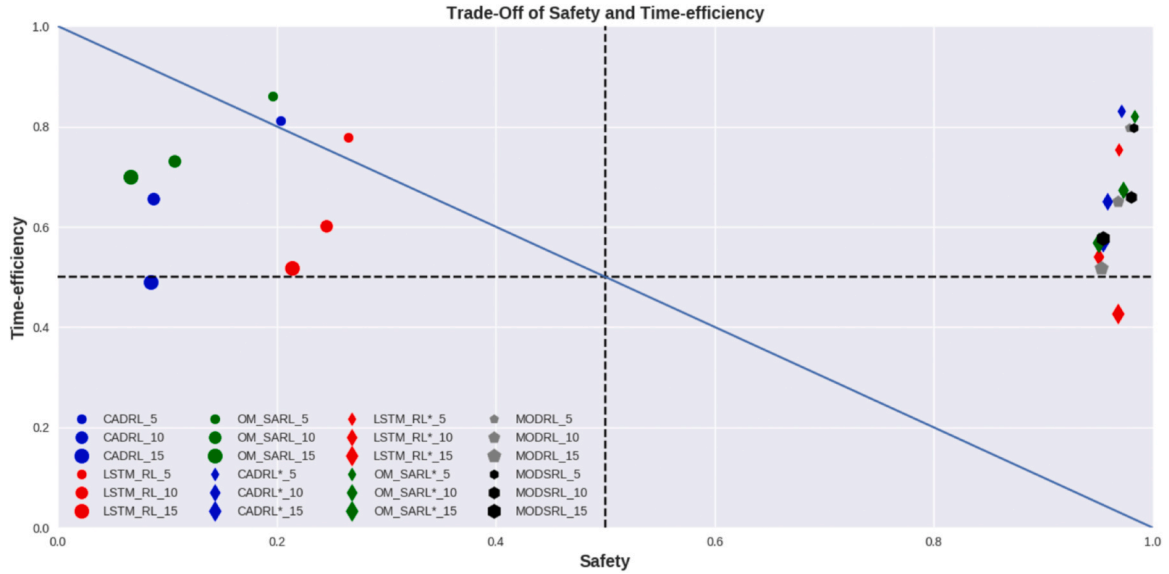


Fig. 11. Trade-off between safety and time efficiency for various methods in a fixed environment.

average values of safety-efficiency (SE) and time-efficiency (TE) over 500 runs for MODSRL, MODRL, CADRL*, LSTM-RL*, OM-SARL*, and the original deep reinforcement learning methods (CADRL, LSTM-RL, OM-SARL), respectively. As shown in Fig. 11, the original deep reinforcement learning methods have good performance in terms of time efficiency, in which CADRL, LSTM-RL, and OM-SARL have TE rates over

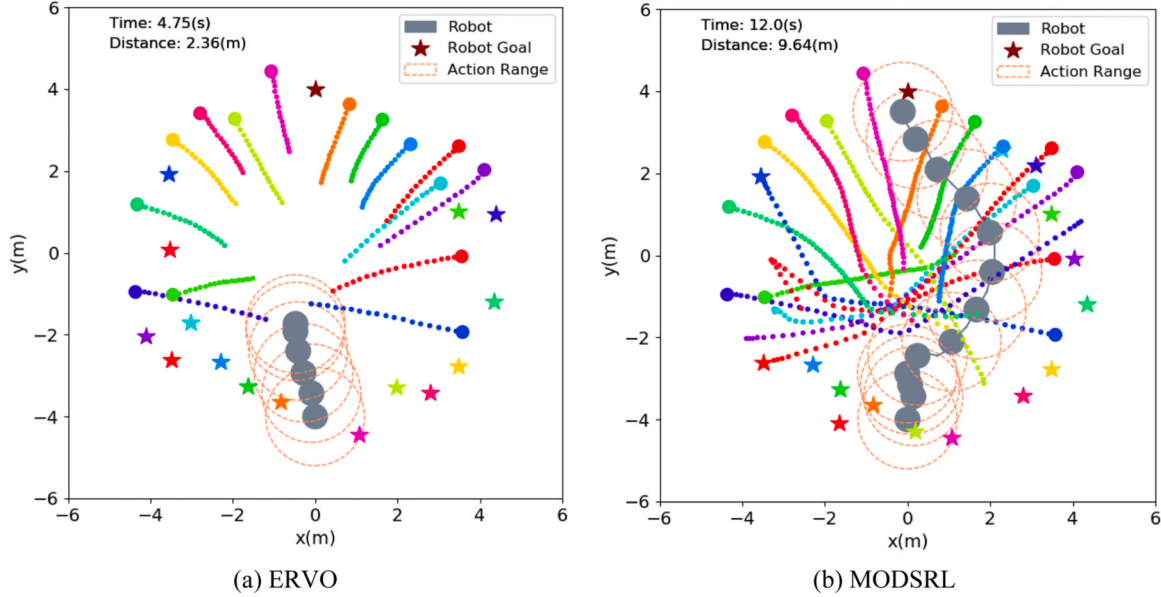
50% in the environments with 5 and 10 humans, and only CADRL has slightly lower TE rate in the environment with 15 humans. However, their safety-efficiency SE values are all below 50% for all three methods. CADRL*, LSTM-RL*, and OM-SARL*, on the other hand, have both safety and time efficiency values greater than 50% in the environments with 5, 10, and 15 humans with the only exception of LSTM-RL*,

Table 2

Navigation results for different numbers of humans in a random environment.

Method	Success Rate $\uparrow$			Collision Rate $\downarrow$			Timeout Rate $\downarrow$			Navigation Time $\downarrow$			Navigation Distance $\downarrow$		
	Human Numbers														
	5	10	15	5	10	15	5	10	15	5	10	15	5	10	15
SFM[20]	0.630	0.220	0.024	0.274	0.346	0.354	0.096	0.434	0.622	17.90	21.18	22.23	11.86	12.59	14.51
ERVO[15]	0.780	0.672	0.516	0.218	0.300	0.434	0.002	0.028	0.050	11.71	14.29	16.26	9.16	9.96	10.56
CADRL*[9]	0.710	0.446	0.344	0.290	0.530	0.648	0.000	0.024	0.008	11.07	13.40	14.81	8.63	9.20	10.56
LSTM-RL*[11]	0.966	0.272	0.090	0.030	0.166	0.396	0.004	0.562	0.514	12.99	17.78	17.63	8.92	12.13	12.20
OM-SARL*[8]	0.978	0.014	0.186	0.020	0.072	0.102	0.002	0.914	0.712	11.12	14.89	14.73	8.87	12.79	10.47
MODRL (Ours)	0.980	0.642	0.366	0.018	0.070	0.130	0.002	0.288	0.504	11.57	16.34	16.24	8.92	11.10	10.42
MODSRL (Ours)	0.996	0.836	0.530	0.004	0.036	0.088	0.000	0.128	0.382	11.37	16.80	17.76	8.77	11.70	10.70

*:Modified to incorporate the proposed multi-reward function.

**Fig. 12.** Navigation trajectories of ERVO and MODSRL with 15 humans.

indicating that the proposed multi-objective reward function successfully improves both safety and time efficiency. Additionally, MODSRL and MODRL achieve the best trade-off between safety and time efficiency, with values in the optimal region for all environments with various human numbers. While OM-SARL* marginally outperforms MODSRL and MODRL in the environment with five humans, MODSRL and MODRL have a better overall performance than the other methods, demonstrating their excellent balance between safety and time efficiency.

The results of robot navigation in random environments for various methods are shown in Table 2. Compared to the traditional methods in an environment with five humans, MODSRL outperforms ERVO and SFM across all five metrics. As the number of humans increases, ERVO and SFM exhibit a gradual decline in success rate along with an increase in collision rate and timeout rate. While in environments with 10 and 15 humans, ERVO outperforms MODSRL in terms of timeout rate, navigation time, and distance. However, it comes at the expense of success rate and collision rate. Notably, as indicated in Table 2, the collision rate increases significantly by 26.4% and 34.6%. As shown in Fig. 12. (a), traditional methods encounter the problem of robot freezing and have a higher probability of collisions in denser crowd environments. On the other hand, with MODSRL, the robot can safely and efficiently reach its destination as shown in Fig. 12. (b).

As shown in Table 2, compared to the learning methods in environments with 5 and 10 humans, CADRL* outperforms OM-SARL* and LSTM-RL* in terms of navigation distance and time but has the highest

collision rate. As a whole, in environments with 5 and 10 humans, MODSRL consistently outperforms the other methods in terms of success rate, collision rate, and timeout rate. Nevertheless, in an environment with ten humans, CADRL* achieves the lowest timeout rate, leading to the shortest navigation distance and better navigation time albeit at the cost of safety. In the environment with 15 humans, MODSRL surpasses CADRL* in success rate by 18.6% and reduces the collision rate by 56%, as indicated in Table 2. This demonstrates that the robot can reach its target more effectively while maintaining safety. Fig. 13 displays the trajectory results of the five learning based methods. Except for MODSRL, in a random environment, LSTM-RL*, OM-SARL*, and MODRL all exhibit delays at the beginning of navigation, as shown in Fig. 13. (b) - (d). Although CADRL* performs the best in navigation time and distance, as shown in Fig. 13. (a), it has the highest collision rate, exceeding MODSRL by 28.6%, thereby compromising safety.

As an ablation analysis, MODSRL demonstrates superior performance compared to MODRL in terms of success rate, collision rate, and time efficiency, as shown in Table 2. In the environment with five humans, MODSRL exhibits a navigation time that is 0.20 s shorter and a navigation distance that is 0.15 m shorter than MODRL, as shown in Table 2. This improvement is attributed to the inclusion of the selection mechanism, which assists the robot to reduce less important interactions with humans, enabling the robot to make better decisions and reduce initial motion delays. While this method may slightly increase navigation distance in a fixed environment, in a random environment, both navigation time and distance of MODSRL are superior to MODRL, as

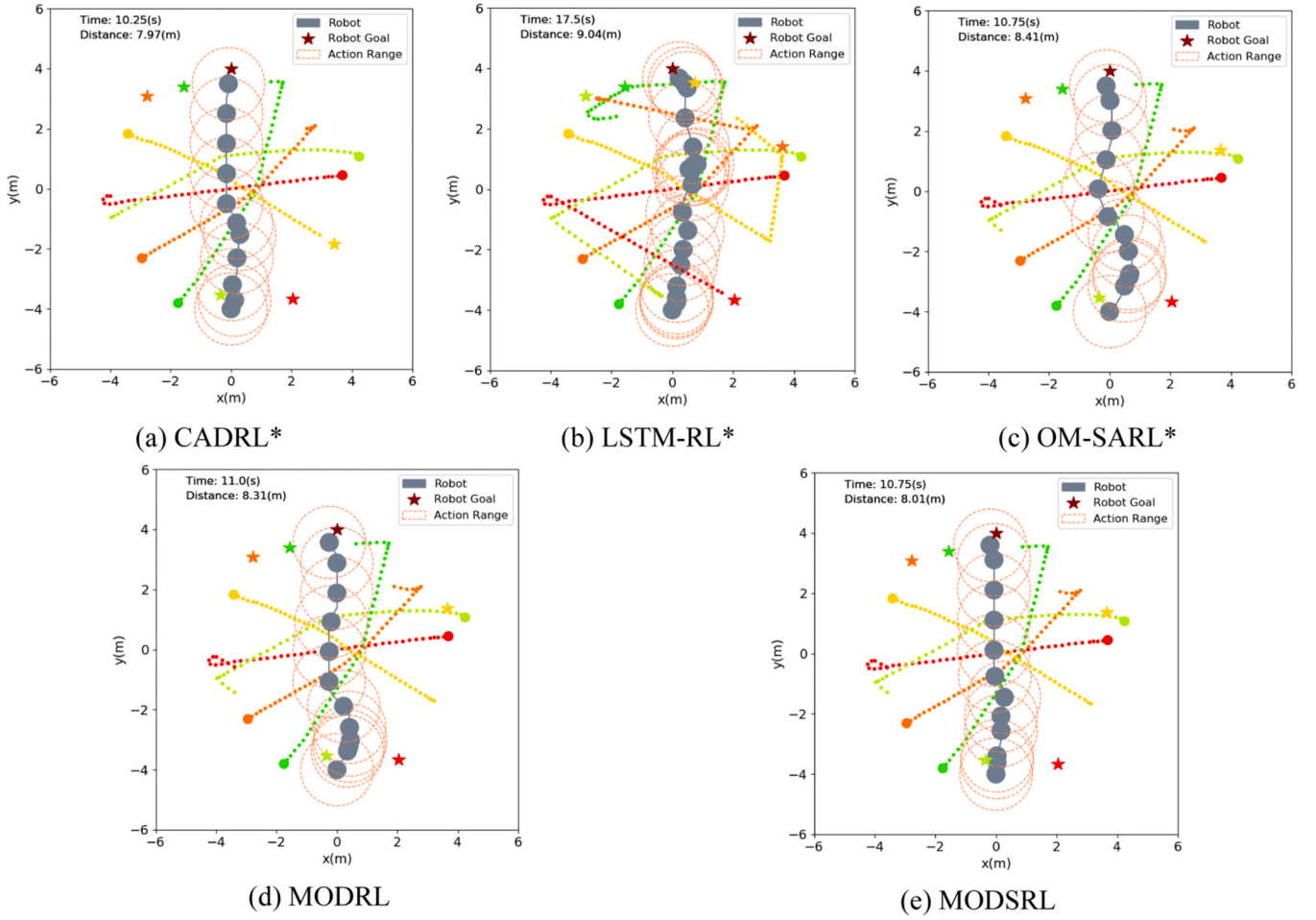


Fig. 13. Navigation trajectories of five learning methods with five humans in the same environment.

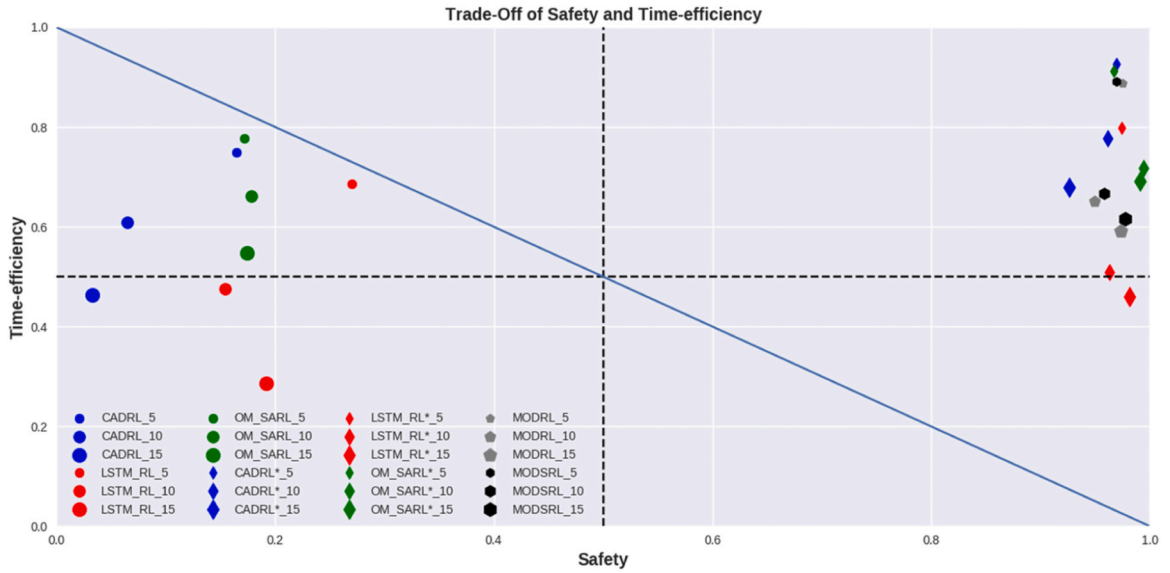


Fig. 14. Trade-off between safety and time efficiency for various methods in a random environment.

shown in Fig. 13. (d) and (e).

In random environments, as depicted in Fig. 14, it can be observed that the original deep reinforcement learning methods experience a decrease in overall data distribution in terms of time efficiency

compared to fixed environments, but they still perform well. In the environment with five humans, the values for CADRL, LSTM-RL, and OM-SARL all exceed 50%. In an environment with ten humans, the values for CADRL and OM-SARL remain above 50%, while LSTM-RL falls

below 50%. As the number of individuals increases, in the environment with 15 humans, only OM-SARL exhibits time efficiency values above 50%, with the other methods falling below 50%. Similarly, despite their good performance in terms of time efficiency, they sacrifice safety, as all three methods have safety efficiency values below 50%. However, CADRL*, LSTM-RL*, and OM-SARL* achieve both safety and time efficiency values greater than 50% in the five and 10-human environments, positioning them in the optimal region. LSTM-RL* is the only method with time efficiency below 50%, aligning with the results in the fixed environment. There is a slight improvement in time efficiency for LSTM-RL*. This indicates that the inclusion of the proposed multi-objective reward function successfully enhances both safety and time efficiency. MODSRL and MODRL achieve safety and time efficiency values in the optimal region for 5, 10, and 15-human environments. While in the environment with five humans, CADRL* and OM-SARL* outperform MODSRL and MODRL in time efficiency, they sacrifice a slight degree of safety. Overall, MODSRL and MODRL outperform the other three methods, demonstrating their excellent balance between safety and time efficiency.

6. Conclusion

In this paper, a multi-objective crowd-aware robot navigation system is introduced, utilizing deep reinforcement learning. The system incorporates an attention extraction module to consider interactions between humans and the robot, along with a double selection module to enhance the robot's ability to make more informed motion decisions. In response to the hesitation challenge encountered at the beginning during navigation in crowd environments, we developed a dual-selection attention module, which empowers the robot to make effective decisions, thereby mitigating the hesitation problem. In light of the experimental results, the proposed method demonstrates satisfactory performance via the design of multiple reward functions for robot navigation in crowd environments, outperforming other baseline methods. There exist, however, some limitations of the proposed approach. First, the simulation environment assumes a full viewing angle from the robot, indicating the robot's sensors have a 360-degree field of vision. In a real-world scenario, this setup can incur higher costs due to the need for additional cameras to be mounted on the robot. Second, the presence of only dynamic obstacles such as humans, without any static obstacles in the simulation environment, would pose a challenge in a real-world setting. As future work of this paper, we plan to explore alternative methods to address the issue of non-convex reward functions. Furthermore, the trained architecture developed in this paper will be implemented on a real robot to assess its performance in a real crowd environment, where object detection and human tracking techniques will be used to obtain the humans' states while localization methods together with odometry information will be utilized to measure the robot's state.

CRedit authorship contribution statement

Chien-Lun Cheng: Methodology, Software, Investigation, Visualization, Writing - Original Draft. Chen-Chien Hsu: Conceptualization, Supervision, Project administration, Writing- Reviewing and Editing, Funding acquisition. Saeed Saeedvand: Validation, Writing - Review & Editing. Jun-Hyung Jo: Validation, Writing- Reviewing and Editing.

Declaration of Competing Interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Chen-Chien Hsu reports financial support was provided by National Science and Technology Council.

Data Availability

Data will be made available on request.

Acknowledgments

This work was supported in part by the Chinese Language and Technology Center of National Taiwan Normal University (NTNU) from The Featured Areas Research Center Program within the Framework of the Higher Education Sprout Project by the Ministry of Education (MOE) in Taiwan, and Ministry of Science and Technology, Taiwan, under Grants no. MOST 110-2634-F-003-006 and MOST 110-2634-F-003-007 through Pervasive Artificial Intelligence Research (PAIR) Labs.

References

- [1] R.C. Arkin, R.R. Murphy, Autonomous navigation in a manufacturing environment, *IEEE Trans. Robot. Autom.* 6 (4) (1990) 445–454.
- [2] J. Wang, M.Q. Meng, Socially compliant path planning for robotic autonomous luggage trolley collection at airports, *Sensors (Basel)* 9 (12) (2019) 2759.
- [3] G. Grisetti, C. Stachniss, W. Burgard, Improved techniques for grid mapping with rao-blackwellized particle filters, *IEEE Trans. Robot.* 23 (1) (2007) 34–46.
- [4] F. Dellaert, D. Fox, W. Burgard, S. Thrun, Monte Carlo localization for mobile robots, *Proc. 1999 IEEE Int. Conf. Robot. Autom. Detroit MI U. S. A.* (1999) 1322–1328.
- [5] A. Dayal, L.R. Cenkeramaddi, A. Jha, Reward criteria impact on the performance of reinforcement learning agent for autonomous navigation, in: *Applied Soft Computing*, 126, Elsevier BV, 2022, 109241.
- [6] L. Ruotsalainen, A. Morrison, M. Mäkelä, J. Rantanen, N. Sokolova, Improving computer vision-based perception for collaborative indoor navigation, *IEEE Sens. J.* 22 (6) (2022) 4816–4826.
- [7] S. Wen, Z. Wen, D. Zhang, H. Zhang, T. Wang, A multi-robot path-planning algorithm for autonomous navigation using meta-reinforcement learning based on transfer learning, in: *Applied Soft Computing*, 110, Elsevier BV, 2021, 107605.
- [8] C. Chen, Y. Liu, S. Kreiss, A. Alahi, Crowd-robot interaction: crowd-aware robot navigation with attention-based deep reinforcement learning, *arXiv* (2018).
- [9] Y.F. Chen, M. Liu, M. Everett, J.P. How, Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning, *arXiv* (2016).
- [10] Y.F. Chen, M. Everett, M. Liu, J.P. How, Socially aware motion planning with deep reinforcement learning, *arXiv* (2017).
- [11] M. Everett, Y.F. Chen, J.P. How, Motion planning among dynamic, decision-making agents with deep reinforcement learning, *arXiv* (2018).
- [12] J. van den Berg, S.J. Guy, M. Lin, D. Manocha, Reciprocal n-body collision avoidance, in: C. Pradaliar, R. Siegwart, G. Hirzinger (Eds.), *Robotics Research. Springer Tracts in Advanced Robotics*, vol. 70, Springer, Berlin, Heidelberg, 2011.
- [13] H. Zeng, R. Hu, X. Huang, Z. Peng, Robot navigation in crowd based on dual social attention deep reinforcement learning, *Math. Probl. Eng.* 2021 (2021) 1–11.
- [14] Yixian Lin, Shibo Song, Junfeng Yao, et al. Robot Navigation in Crowd via Deep Reinforcement Learning, 27 June 2022, PREPRINT (Version 1) available at Research Square.
- [15] M. Nishimura, R. Yonetani, L2B: learning to balance the safety-efficiency trade-off in interactive crowd-aware robot navigation, *arXiv* (2020).
- [16] H. Cheng, H. Chen, Y. Liu, Topological indoor localization & navigation for autonomous industrial mobile manipulator, 2012 11th Int. Conf. Mach. Learn. Appl. Boca Raton FL U. S. A (2012) 238–243.
- [17] C. Wang, J. Wang, C. Li, D. Ho, J. Cheng, T. Yan, L. Meng, M.Q.H. Meng, Safe and robust mobile robot navigation in uneven indoor environments, *Sens. (Basel)* 19 (13) (2019) 2993.
- [18] K. Zhu, T. Zhang, Deep reinforcement learning based mobile robot navigation a review, *Tsinghua Sci. Technol.* 26 (5) (2021) 674–691.
- [19] J. van den Berg, Ming Lin, D. Manocha, Reciprocal velocity obstacles for real-time multi-agent navigation, 2008 IEEE Int. Conf. Robot. Autom. (2008) 1928–1935.
- [20] D. Helbing, P. Molnar, "Social Force Model for Pedestrian Dynamics," *arXiv*, 1998.
- [21] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, S. Savarese, Social LSTM: human trajectory prediction in crowded spaces, 2016 IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR) (2016) 961–971.
- [22] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, A. Alahi, Social GAN: socially acceptable trajectories with generative adversarial networks, *arXiv* (2018).
- [23] P. Trautman, A. Krause, Unfreezing the robot: navigation in dense, interacting crowds, 2010 IEEE/RSJ Int. Conf. Intell. Robots Syst. (2010) 797–803.
- [24] L. Kästner, J. Li, Z. Shen, J. Lambrecht, Enhancing navigational safety in crowded environments using semantic-deep-reinforcement-learning-based navigation, *arXiv* (2021).
- [25] S.S. Samsani, On safety and time efficiency enhancement of robot navigation in crowded environment utilizing deep reinforcement learning, *Inst. Electr. Electron. Eng. (IEEE)* (2021).
- [26] K. Van Moffaert, M.M. Drugan, A. Nowé, Scalarized multi-objective reinforcement learning: novel design techniques, 2013 IEEE Symp. . Adapt. Dyn. Program. Reinf. Learn. (ADPRL), Singap. (2013) 191–199.
- [27] T.T. Nguyen, N.D. Nguyen, P. Vamplew, S. Nahavandi, R. Dazeley, C.P. Lim, A multi-objective deep reinforcement learning framework, *arXiv* (2018).

- [28] A. Ramezani Dooraki, D.-J. Lee, A multi-objective reinforcement learning based controller for autonomous navigation in challenging environments, *Machines* 10 (7) (2022) 500.
- [29] C. Yan, G. Chen, Y. Li, F. Sun, Y. Wu, Immune deep reinforcement learning-based path planning for mobile robot in unknown environment, *Appl. Soft Comput.* 145 (2023), 110601.
- [30] Y. Chen, F. Zhao, Y. Lou, Interactive model predictive control for robot navigation in dense crowds, *IEEE Trans. Syst. Man Cybern.: Syst.* 52 (4) (2022) 2289–2301.